Experiment No 6
LAB MANUAL
PART B

Roll No. C-60	**Name: Avdhut Parish Madhavi**
Class: SE-C	**Batch: C2**
Date of Experiment:	**Date of Submission:**
Grade:	

B.1: Implement and trace the following programs. Write the code, compile/run it, and record the output after each task.

1. Run Prim's algorithm by changing the starting vertex from 0 to another vertex and verify whether the total MST cost remains the same.
CODE:

```
#include <stdio.h>
#include <limits.h>
#define V 5
int minKey(int key[], int mstSet[])
{
int min = INT_MAX, minIndex = -1;
for (int v = 0; v < V; v++)
{
if (!mstSet[v] && key[v] < min) {
min = key[v];
minIndex = v;
}
}
return minIndex;
}
void primMST(int graph[V][V], int startVertex)
{
int parent[V], key[V], mstSet[V];
int totalCost = 0;
for (int i = 0; i < V; i++)
{
key[i] = INT_MAX;
mstSet[i] = 0;
}
key[startVertex] = 0;
parent[startVertex] = -1;
for (int count = 0; count < V - 1; count++)
{
int u = minKey(key, mstSet);
 mstSet[u] = 1;
for (int v = 0; v < V; v++)
{
if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v])
```

```
{
parent[v] = u;
key[v] = graph[u][v];
}       }
    }
 printf ("Prim's MST (Star ng Vertex = %d):\n", startVertex);
printf ("Edge\tWeight\n");
for (int i = 0; i < V; i++)
{
if (parent[i] != -1) {
printf("%d - %d\t%d\n", parent[i], i, graph[i][parent[i]]);
totalCost += graph[i][parent[i]];
}
}
printf ("Total cost = %d\n", totalCost);
}
int main()
{
int graph[V][V] = {
{0, 2, 0, 6, 0},
{2, 0, 3, 8, 5},
 {0, 3, 0, 0, 7},
{6, 8, 0, 0, 9},
{0, 5, 7, 9, 0}
};
primMST(graph, 2);
return 0;
}
```

```
Prim's MST (Starting Vertex = 2):
Edge    Weight
1 - 0   2
2 - 1   3
0 - 3   6
1 - 4   5
Total cost = 16
```

2. Run Kruskal's algorithm after entering the graph edges in a different input order and verify that sorting still produces the correct MST.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#define V 5
#define E 7
typedef struct Edge {
```

```
 int src, dest, weight;
 }
Edge;
int parent[V], rankArr[V];
int findSet(int i)
{
if (parent[i] != i)        parent[i] = findSet(parent[i]);
return parent[i];
}
void unionSet(int x, int y)
{
int rootX = findSet(x);
int rootY = findSet(y);
if (rootX != rootY)
{
if (rankArr[rootX] < rankArr[rootY])
 parent[rootX] = rootY;
 else if (rankArr[rootX] > rankArr[rootY])
 parent[rootY] = rootX;
else
{
parent[rootY] = rootX;
rankArr[rootX]++;
}
}
}
int compare(const void *a, const void *b)
{
Edge *e1 = (Edge *)a;
Edge *e2 = (Edge *)b;
return e1->weight - e2->weight;
}
int main()
{
Edge edges[E] = {
{3, 4, 9},
{0, 3, 6},
{1, 4, 5},
{0, 1, 2},
{2, 4, 7},
{1, 2, 3},
{1, 3, 8}
};
for (int i = 0; i < V; i++)
{
parent[i] = i;
rankArr[i] = 0;
}
qsort(edges, E, sizeof(Edge), compare);
int selected = 0, totalCost = 0;
printf ("Kruskal's MST (Shuffled Input Order):\n");
```

```
printf ("Edge\tWeight\n");
for (int i = 0; i < E && selected < V - 1; i++)
{
int u = edges[i].src;
int v = edges[i].dest;
if (findSet(u) != findSet(v))
{
printf ("%d - %d\t%d\n", u, v, edges[i].weight);
totalCost += edges[i].weight;
unionSet(u, v);
selected++;
}
}
printf ("Total cost = %d\n", totalCost);
return 0;
}
```

```
Kruskal's MST (Shuffled Input Order):
Edge    Weight
0 - 1   2
1 - 2   3
1 - 4   5
0 - 3   6
Total cost = 16
```

3. Modify one or more edge weights and observe how the selected MST edges and total cost change.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#define V 5
#define E 7
typedef struct Edge {
 int src, dest, weight;
 }
Edge;
int parent[V], rankArr[V];
int findSet(int i)
{
if (parent[i] != i)        parent[i] = findSet(parent[i]);
return parent[i];
}
void unionSet(int x, int y)
{
int rootX = findSet(x);
```

```
int rootY = findSet(y);
if (rootX != rootY)
{
if (rankArr[rootX] < rankArr[rootY])
 parent[rootX] = rootY;
 else if (rankArr[rootX] > rankArr[rootY])
 parent[rootY] = rootX;
else
{
parent[rootY] = rootX;
rankArr[rootX]++;
}
}
}
int compare(const void *a, const void *b)
{
Edge *e1 = (Edge *)a;
Edge *e2 = (Edge *)b;
return e1->weight - e2->weight;
}
int main()
{
Edge edges[E] = {
{0, 1, 2},
{0, 3, 6},
{1, 2, 10},
{1, 3, 8},
{1, 4, 5},
{2, 7, 4},
{3, 4, 9}
};
for (int i = 0; i < V; i++)
{
parent[i] = i;
rankArr[i] = 0;
}
qsort(edges, E, sizeof(Edge), compare);
int selected = 0, totalCost = 0;
printf ("Kruskal's MST (Shuffled Input Order):\n");
printf ("Edge\tWeight\n");
for (int i = 0; i < E && selected < V - 1; i++)
{
int u = edges[i].src;
int v = edges[i].dest;
if (findSet(u) != findSet(v))
{
printf ("%d - %d\t%d\n", u, v, edges[i].weight);
totalCost += edges[i].weight;
unionSet(u, v);
selected++;
}
```

```
}
printf ("Total cost = %d\n", totalCost);
return 0;
}
```

```
Kruskal's MST (Modified Edge Weight):
Edge    Weight
0 - 1   2
1 - 4   5
0 - 3   6
2 - 4   7
Total cost = 20
```

4. Compare the sequence of edge selection in Prim's and Kruskal's algorithms for the same graph.

CODE:

```
#include <stdio.h>
#include <limits.h>
#define V 5
int minKey(int key[], int mstSet[])
{
int min = INT_MAX, minIndex = -1;
for (int v = 0; v < V; v++)
{
if (!mstSet[v] && key[v] < min) {
min = key[v];
minIndex = v;
}
}
return minIndex;
}
void primMST(int graph[V][V], int startVertex)
{
int parent[V], key[V], mstSet[V];
int totalCost = 0;
for (int i = 0; i < V; i++)
{
key[i] = INT_MAX;
mstSet[i] = 0;
}
key[startVertex] = 0;
parent[startVertex] = -1;
for (int count = 0; count < V - 1; count++)
{
int u = minKey(key, mstSet);
```

```
 mstSet[u] = 1;
for (int v = 0; v < V; v++)
{
if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v])
{
parent[v] = u;
key[v] = graph[u][v];
}       }
    }
 printf ("Prim's MST (Star ng Vertex = %d):\n", startVertex);
printf ("Edge\tWeight\n");
for (int i = 0; i < V; i++)
{
if (parent[i] != -1) {
printf("%d - %d\t%d\n", parent[i], i, graph[i][parent[i]]);
totalCost += graph[i][parent[i]];
}
}
printf ("Total cost = %d\n", totalCost);
}
void unionSet(int x, int y)
{
int rootX = findSet(x);
int rootY = findSet(y);
if (rootX != rootY)
{
if (rankArr[rootX] < rankArr[rootY])
 parent[rootX] = rootY;
 else if (rankArr[rootX] > rankArr[rootY])
 parent[rootY] = rootX;
else
{
parent[rootY] = rootX;
rankArr[rootX]++;
}
}
}
int compare(const void *a, const void *b)
{
Edge *e1 = (Edge *)a;
Edge *e2 = (Edge *)b;
return e1->weight - e2->weight;
}
int main()
{
Edge edges[E] = {
{0, 1, 2},
{0, 3, 6},
{1, 2, 10},
{1, 3, 8},
{1, 4, 5},
```

```
{2, 7, 4},
{3, 4, 9}
};
for (int i = 0; i < V; i++)
{
parent[i] = i;
rankArr[i] = 0;
}

int graph[V][V] = {
{0, 2, 0, 6, 0},
{2, 0, 3, 8, 5},
 {0, 3, 0, 0, 7},
{6, 8, 0, 0, 9},
{0, 5, 7, 9, 0}
};
primMST(graph, 2);
return 0;
}
```

```
Prim's Edge Selection Sequence:
0 - 1 (Weight: 2)
1 - 2 (Weight: 3)
0 - 3 (Weight: 6)
1 - 4 (Weight: 5)
Prim's Total Cost = 16

Kruskal's Edge Selection Sequence:
0 - 1 (Weight: 2)
1 - 2 (Weight: 3)
1 - 4 (Weight: 5)
0 - 3 (Weight: 6)
Kruskal's Total Cost = 16
```

5. Do the time and space complexity analysis of Prim's and Kruskal's algorithms.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
 #include <limits.h>
#define V 5
#define E 7
typedef struct Edge {
```

```
int src, dest, weight;
} Edge;
// Prim’s code with counters
int minKey(int key[], int mstSet[], int *ops)
{
int min = INT_MAX, minIndex = -1;
for (int v = 0; v < V; v++)
{
(*ops)++; // Count comparison
if (!mstSet[v] && key[v] < min)
{
min = key[v];
minIndex = v;
}
}
return minindex;
void primMST(int graph[V][V], int *ops)
{
int parent[V], key[V], mstSet[V];
int totalCost = 0;
for (int i = 0; i < V; i++)
{
key[i] = INT_MAX;
mstSet[i] = 0;
}
key[0] = 0;
parent[0] = -1;
for (int count = 0; count < V - 1; count++)
{
int u = minKey(key, mstSet, ops);
mstSet[u] = 1;
for (int v = 0; v < V; v++)
{
(*ops)++; // Count adjacency check
if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v])
{
parent[v] = u;
key[v] = graph[u][v];
}
}
}
printf("Prim's Total Cost = ");
for (int i = 1; i < V; i++)
{
totalCost += graph[i][parent[i]];
}
printf("%d\n", totalCost);
}  // Kruskal’s code with counters
int parent[V], rankArr[V];
int kruskalOps = 0;
int findSet(int i)
```

```
{
kruskalOps++;
if (parent[i] != i)
parent[i] = findSet(parent[i]);
return parent[i];
}
void unionSet(int x, int y)
{
int rootX = findSet(x);
int rootY = findSet(y);
if (rootX != rootY)
{
if (rankArr[rootX] < rankArr[rootY])
parent[rootX] = rootY;
else if (rankArr[rootX] > rankArr[rootY])
parent[rootY] = rootX;
else
{
parent[rootY] = rootX;
rankArr[rootX]++;
}
}
}
 int compare(const void *a, const void *b)
{
kruskalOps++;
Edge *e1 = (Edge *)a;
Edge *e2 = (Edge *)b;
return e1->weight - e2->weight;
}
void kruskalMST(Edge edges[])
{
for (int i = 0; i < V; i++)
{
parent[i] = i;
rankArr[i] = 0;
}
qsort(edges, E, sizeof(Edge), compare);
int selected = 0, totalCost = 0;
for (int i = 0; i < E && selected < V - 1; i++)
{
int u = edges[i].src;
int v = edges[i].dest;
if (findSet(u) != findSet(v))
{
totalCost += edges[i].weight;
unionSet(u, v);
selected++;
}
}
printf("Kruskal's Total Cost = %d\n", totalCost);
```

```
}
int main()
{
int graph[V][V] = {
{0, 2, 0, 6, 0},
{2, 0, 3, 8, 5},
{0, 3, 0, 0, 7},
{6, 8, 0, 0, 9},
{0, 5, 7, 9, 0}
};
Edge edges[E] = {
{0, 1, 2}, {0, 3, 6}, {1, 2, 3}, {1, 3, 8},        {1, 4, 5}, {2, 4, 7}, {3, 4, 9}
};

  printf ("\n~ Complexity Analysis ~ (V=%d, E=%d) \n", V, E);
printf("Prim's Opera ons (approx): %d\n", primOps);
printf("  -> Theore cal Time: O(V^2) = O(%d)\n", V*V);
printf("  -> Space: O(V) = O(%d)\n", V);
printf("Kruskal's Opera ons (approx): %d\n", kruskalOps);
printf("  -> Theore cal Time: O(E log E) = O(%d * log %d)\n", E, E);
printf("  -> Space: O(V + E) = O(%d + %d)\n", V, E);
printf("\nObserva on:\n");
printf("Prim's is be er for dense graphs (many edges).\n");
printf("Kruskal's is be er for sparse graphs (few edges).\n");
return 0;
}
```

```
Prim's Total Cost = 16
Kruskal's Total Cost = 16

~ Complexity Analysis ~ (V=5, E=7)
Prim's Operations (approx): 40
  -> Theoretical Time: O(V^2) = O(25)
  -> Space: O(V) = O(5)
Kruskal's Operations (approx): 31
  -> Theoretical Time: O(E log E) = O(7 * log 7)
  -> Space: O(V + E) = O(5 + 7)

Observation:
Prim's is better for dense graphs (many edges).
Kruskal's is better for sparse graphs (few edges).
```